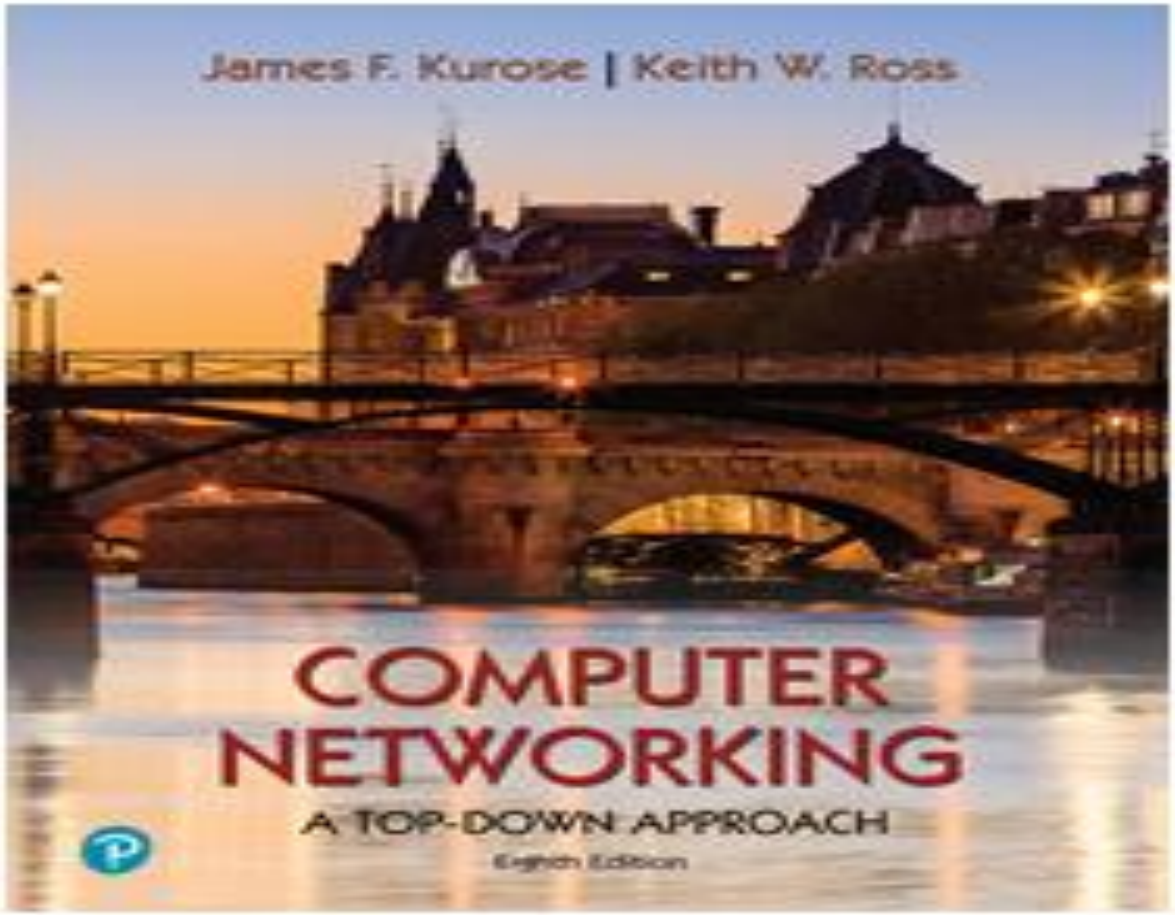


Computer Networking: A Top-Down Approach

8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Chapter 3 Transport Layer

Residing between the application and network layers, the transport layer is a central piece of the layered network architecture.
It has the critical role of providing communication services directly to the application processes running on different hosts.

as usual, particular emphasis will be given to Internet protocols, in particular the TCP and UDP transport-layer protocols.

We'll begin by discussing the relationship between the transport and network layers.

We'll then return to principles and confront one of the most fundamental problems in computer networking—how two entities can communicate reliably over a medium that may lose and corrupt data.
Through a series of increasingly complicated (and realistic!) scenarios, we'll build up an array of techniques that transport protocols use to solve this problem.

We'll then show how these principles are embodied in TCP, the Internet's connection-oriented transport protocol.

We'll next move on to a second fundamentally important problem in networking—controlling the transmission rate of transport-layer entities in order to avoid, or recover from, congestion within the network.
We'll consider the causes and consequences of congestion, as well as commonly used congestion-control techniques.
After obtaining a solid understanding of the issues behind congestion control, we'll study TCP's approach to congestion control.

3.1 Introduction and Transport-Layer Services

In the previous two chapters, we touched on the role of the transport layer and the services that it provides.
Let's quickly review what we have already learned about the transport layer.
A transport-layer protocol provides for **logical communication** between application processes running on different hosts.
By *logical communication*, we mean that from an application's perspective, it is as if the hosts running the processes were directly connected;
in reality, the hosts may be on opposite sides of the planet, connected via numerous routers and a wide range of link types.

Application processes use the logical communication provided by the transport layer to send messages to each other, free from the worry of the details of the physical infrastructure used to carry these messages.

On the sending side, the transport layer converts the application-layer messages it receives from a sending application process into transport-layer packets, known as transport-layer **segments** in Internet terminology.
This is done by (possibly) breaking the application messages into smaller chunks and adding a transport-layer header to each chunk to create the transport-layer segment.

The transport layer then passes the segment to the network layer at the sending end system, where the segment is encapsulated within a network-layer packet (a datagram) and sent to the destination.

It's important to note that network routers act only on the network-layer fields of the datagram; that is, they do not examine the fields of the transport-layer segment encapsulated with the datagram.

On the receiving side, the network layer extracts the transport-layer segment from the datagram and passes the segment up to the transport layer.
The transport layer then processes the received segment, making the data in the segment available to the receiving application.

More than one transport-layer protocol may be available to network applications.
For example, the Internet has two protocols—TCP and UDP.
Each of these protocols provides a different set of transport-layer services to the invoking application.

3.1.1 Relationship Between Transport and Network Layers

Whereas a transport-layer protocol provides logical communication between *processes* running on different hosts, a network-layer protocol provides logical-communication between *hosts*.
This distinction is subtle but important.

transport-layer protocols live in the end systems.
Within an end system, a transport protocol moves messages from application processes to the network edge (that is, the network layer) and vice versa, but it doesn't have any say about how the messages are moved within the network core.

In fact, intermediate routers neither act on, nor recognize, any information that the transport layer may have added to the application messages.

a computer network may make available multiple transport protocols, with each protocol offering a different service model to applications.

the services that a transport protocol can provide are often constrained by the service model of the underlying network-layer protocol.
If the network-layer protocol cannot provide delay or bandwidth guarantees for transport-layer segments sent between hosts, then the transport-layer protocol cannot provide delay or bandwidth guarantees for application messages sent between processes.

Nevertheless, certain services *can* be offered by a transport protocol even when the underlying network protocol doesn't offer the corresponding service at the network layer.

For example, as we'll see in this chapter, a transport protocol can offer reliable data transfer service to an application even when the underlying network protocol is unreliable, that is, even when the network protocol loses, garbles, or duplicates packets.

As another example (which we'll explore in Chapter 8 when we discuss network security), a transport protocol can use encryption to guarantee that application messages are not read by intruders, even when the network layer cannot guarantee the confidentiality of transport-layer segments.

3.1.2 Overview of the Transport Layer in the Internet

Recall that the Internet makes two distinct transport-layer protocols available to the application layer.

One of these protocols is **UDP** (User Datagram Protocol), which provides an unreliable, connectionless service to the invoking application.

The second of these protocols is **TCP** (Transmission Control Protocol), which provides a reliable, connection-oriented service to the invoking application.

When designing a network application, the application developer must specify one of these two transport protocols.

To simplify terminology, we refer to the transport-layer packet as a *segment*.

We mention, however, that the Internet literature (for example, the RFCs) also refers to the transport-layer packet for TCP as a segment but often refers to the packet for UDP as a datagram.

However, this same Internet literature also uses the term *datagram* for the network-layer packet! For an introductory book on computer networking such as this, we believe that it is less confusing to refer to both TCP and UDP packets as segments, and reserve the term *datagram* for the network-layer packet.

Before proceeding with our brief introduction of UDP and TCP, it will be useful to say a few words about the Internet's network layer.
(We'll learn about the network layer in detail in Chapters 4 and 5.)
The Internet's network-layer protocol has a name—IP, for Internet Protocol.
IP provides logical communication between hosts.
The IP service model is a **best-effort delivery service**.
This means that IP makes its "best effort" to deliver segments between communicating hosts, *but it makes no guarantees.*
In particular, it does not guarantee segment delivery, it does not guarantee orderly delivery of segments, and it does not guarantee the integrity of the data in the segments.
For these reasons, IP is said to be an **unreliable service**.

We also mention here that every host has at least one network-layer address, a so-called IP address. We'll examine IP addressing in detail in Chapter 4; for this chapter we need only keep in mind that *each host has an IP address*.

Having taken a glimpse at the IP service model, let's now summarize the service models provided by UDP and TCP.
The most fundamental responsibility of UDP and TCP is to extend IP's delivery service between two end systems to a delivery service between two processes running on the end systems.
Extending host-to-host delivery to process-to-process delivery is called **transport-layer multiplexing** and **demultiplexing**.
We'll discuss transport-layer multiplexing and demultiplexing in the next section.

UDP and TCP also provide integrity checking by including error-detection fields in their segments' headers.

These two minimal transport-layer services—process-to-process data delivery and error checking—are the only two services that UDP provides! In particular, like IP, UDP is an unreliable service—it does not guarantee that data sent by one process will arrive intact (or at all!) to the destination process.
UDP is discussed in detail in Section 3.3.

TCP, on the other hand, offers several additional services to applications.
First and foremost, it provides **reliable data transfer**.
Using flow control, sequence numbers, acknowledgments, and timers (techniques we'll explore in detail in this chapter), TCP ensures that data is delivered from sending process to receiving process, correctly and in order.
TCP thus converts IP's unreliable service between end systems into a reliable data transport service between processes.
TCP also provides **congestion control**.

Loosely speaking, TCP congestion control prevents any one TCP connection from swamping the links and routers between communicating hosts with an excessive amount of traffic.

UDP traffic, on the other hand, is unregulated.
An application using UDP transport can send at any rate it pleases, for as long as it pleases.